

URL

UNIVERSAL RESOURCE LOCATOR

INTRODUZIONE

Principi architetturali del web

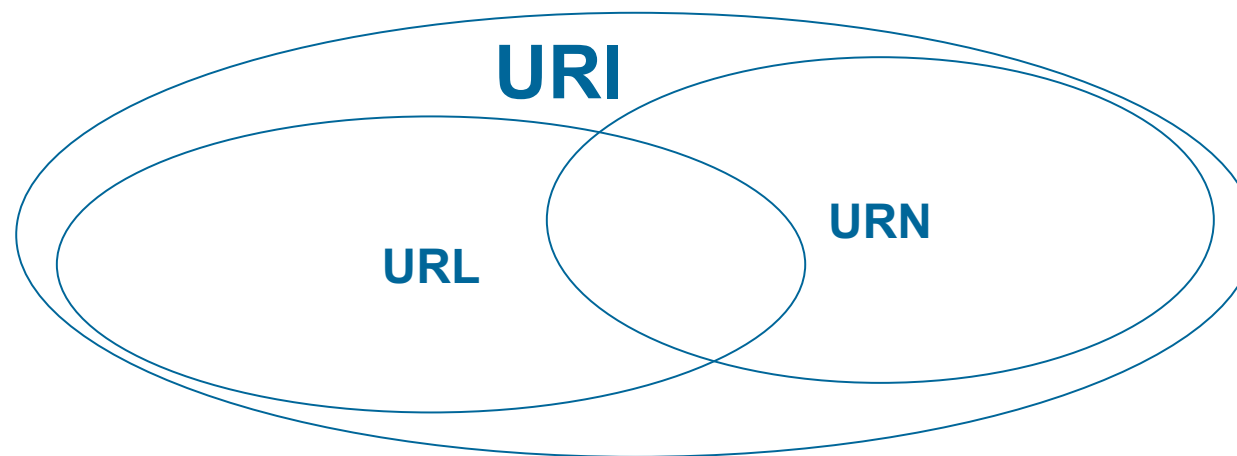
- ❑ **Identificazione:** Il web è uno spazio informativo (per umani e applicazioni) in cui ogni elemento di interesse è chiamato risorsa ed è identificato da un identificatore globale chiamato URI.
- ❑ **Interazione:** Il protocollo HTTP permette di scambiare messaggi su una rete (es. TCP/IP)
- ❑ **Formato:** la disponibilità di apposite applicazioni sul computer ricevente e la corretta identificazione del formato dati con cui la risorsa è stata comunicata permette di accedere al suo contenuto in maniera chiara e non ambigua. I formati sono molti, tra cui HTML, PNG, RDF, ecc.)

URI (1)

- ❑ Gli URI (Uniform Resource Identifier) sono una sintassi usata nel web per definire i nomi e gli indirizzi di oggetti (risorse) su Internet.
- ❑ Questi oggetti sono considerati accessibili tramite l'utilizzo di protocolli esistenti, inventati appositamente, o ancora da inventare.
- ❑ Gli URI si orientano a risolvere il problema di creare un meccanismo ed una sintassi di accesso unificata alle risorse di dati disponibili via rete.
- ❑ Tutte le istruzioni d'accesso ai vari specifici oggetti disponibili secondo un dato protocollo sono codificate come una stringa di indirizzo

URI (2)

- ❑ Gli Uniform Resource Identifier (**URI**) sono, per definizione:
 - Uniform Resource Locator (**URL**): sintassi che contiene informazioni immediatamente utilizzabili per accedere alla risorsa (ad esempio, **il suo indirizzo di rete**)
 - Uniform Resource Names (**URN**): sintassi che permette una etichettatura permanente e non ripudiabile della risorsa, indipendentemente dal riportare informazioni sull'accesso (es. **isbn:0-395-36341-1**)



URL

- ❑ URL = **U**niversal **R**esource **L**ocator
- ❑ tutte le istruzioni d'accesso alle varie specifiche risorse disponibili in rete (secondo un dato protocollo) sono codificate come una stringa di indirizzo
- ❑ Esempio:
 - `http://www.google.com/search?hl=en&lr=&ie=UTF-8&oe=UTF-8&q=alfa+romeo`

Esigenza di identificatori

□ URL:

- verosimilmente, il fattore determinante per il successo del web

□ attraverso gli URL:

- nel web si è in grado di identificare risorse accessibili non solo tramite lo specifico protocollo, HTTP, ma anche tramite tutti gli altri protocolli esistenti (FTP, Telnet, etc.)

□ punto principale:

- sintassi universale, indipendente dal protocollo e facilmente memorizzabile o scambiabile con cui identificare le risorse di rete

Il concetto di risorsa

- ❑ gli URL sono pensati per essere indipendenti dal meccanismo di memorizzazione effettiva
- ❑ anche se molti URL fanno riferimento a file memorizzati in un file system gerarchico, questo non è necessario:
 - potrebbe essere in un file system non gerarchico
 - potrebbe essere in un database, e l'URL essere la chiave di ricerca
 - potrebbe essere il risultato dell'elaborazione di un'applicazione, e l'URL contenere i parametri di elaborazione.
- ❑ per questo si usa il termine Risorsa, invece che File, e si fornisce una sintassi indipendente dal sistema effettivo di memorizzazione. Mai assumere che si stia lavorando con un file system!

Organizzazione degli URI (1)

□ Un URI è diviso nelle parti seguenti:

- URI = schema : [// authority] path [? query] [# fragment]

□ Esempi:

- <http://www.ietf.org/rfc/rfc2396.txt>
- <ftp://ftp.is.co.za/rfc/rfc1808.txt>
- <mailto:John.Doe@example.com>
- <file:///Documenti/corsi/tw04/slides/l1.html>
- <news:comp.infosystems.www.servers.unix>
- <telnet://melvyl.ucop.edu/>

Organizzazione degli URI (2)

- ❑ Lo schema (negli URL è il protocollo) é identificato da una stringa arbitraria (ma registrata presso IANA) usata come prefisso.
- ❑ La presenza esplicita di un'authority individua un'organizzazione gerarchica dello spazio dei nomi a cui sono delegati i nomi (che possono essere ulteriormente delegati). L'autorità è a sua volta divisa in:

`authority = [userinfo @] host [: port]`

- ❑ La parte userinfo non deve essere presente se lo schema non prevede identificazione personale. La parte host è o un nome di dominio o un indirizzo IP. La port può essere omessa se ci si riferisce ad una well-known port (per http è la porta 80).

Organizzazione degli URI (3)

- ❑ **path:** parte identificativa della risorsa all'interno dello spazio di nomi identificato dallo schema e (se esistente) dalla authority
 - la parte authority evidenzia uno spazio di nomi gerarchico
 - in questi casi la parte path è divisa in blocchi separati da slash “/”, ciascuno dei quali è un componente del path organizzato in gerarchia
 - in questo caso diventano significativi gli pseudo componenti “.” e “..”
- ❑ **query:** ulteriore specificazione della risorsa all'interno dello spazio di nomi identificato dallo schema e dall'URI precedente
 - di solito sono parametri passati all'URI (un processo) per specificare un risultato dinamico (es. l'output di una query su un motore di ricerca)
 - è tutto quello che sta dopo il punto interrogativo e prima del carattere hash
 - tipicamente ha la forma `nome1=valore1&nome2=valore+in+molte+parole`
- ❑ **fragment:** individua una risorsa secondaria (in molti casi un frammento) della risorsa primaria.
 - è tutta la parte che sta dopo al carattere di hash “#”.

Caratteri ammessi /1

- ❑ i caratteri degli URL sono

`curl : unreserved | reserved | escaped`

- ❑ i caratteri non riservati sono alfanumerici e alcuni caratteri di punteggiatura privi di ambiguità

`unreserved: uppercase | lowercase | digit | punctuation`

`punctuation: -_ . ! ~ * ' ( )`

- ❑ i caratteri riservati sono caratteri che hanno delle funzioni particolari in uno o più schemi di URL. In questo caso vanno usati direttamente quando assolvono alle loro funzioni, e escaped quando sono invece parte della stringa identificativa naturale

`reserved: ; / ? : @ & = + $ ,`

Caratteri ammessi /2

- ❑ I caratteri escaped fanno riferimento alle seguenti tipologie di caratteri:
 - I caratteri non ASCII
 - I caratteri di controllo: ASCII < 32
 - I caratteri: { } | \ ^ [] `
 - I delimitatori: spazio < > # % "

- ❑ In questo caso i caratteri vanno posti in maniera escaped, secondo la seguente sintassi:
 - %XX, dove XX è il codice esadecimale del carattere

Caratteri riservati /1

□ “%”

- é il codice di escape, e serve per l'utilizzo di caratteri particolari nell'URL, precedendone il codice esadecimale.
Es., per utilizzare un carattere “%” nel'URL bisogna usare la stringa “%25”

□ /

- Il carattere “/” é utilizzato unicamente per l'identificazione di sottoparti di uno schema gerarchico, e non può essere usato per altri scopi

□ .

- Il punto singolo “.” o doppio “..” hanno anch'essi un significato riservato; usati per indicare risorse allo stesso livello o al livello gerarchico superiore

Caratteri riservati /2

□

- il carattere di hash “#” serve per delimitare l’URL di un oggetto da un identificatore di un frammento interno alla risorsa considerata

□ ?

- Il punto interrogativo “?” serve per separare l’URL di una risorsa cui è possibile passare dei dati (ad esempio, un’applicazione software lato server)

□ +

- all’interno della parte dati, il segno più “+” è usato al posto dello spazio (mai usato)

Caratteri riservati /3

- Esempi di richieste di risorse di differente natura:

```
http://ing.univaq.it/cicerone/prova.html#sezione1  
http://ing.univaq.it/cicerone/saluto.php?nome=Mario  
http://ing.univaq.it/cicerone/saluto.php?nome=Mario+Rossi
```

Query verso Google:

- chiave di ricerca: "alfa romeo"
- URL: `http://www.google.com/search?hl=en&lr=&ie=UTF-8&oe=UTF-8&q=alfa+romeo`
- parametri: `hl`, `lr`, `ie`, `oe`, e `q`
- `q` rappresenta l'effettiva chiave di ricerca, "alfa romeo"

Caratteri riservati /4

❑ I due URL

- `http://www.alpha.com/a/b/c/d`
- `http://www.alpha.com/a/b/c%2Fd`

❑ non sono uguali, perché, benché il codice esadecimale corrisponda al carattere “/”, nel primo caso esso ha significato gerarchico, e nel secondo fa parte del nome dell’ultima sottoparte della gerarchia, “c/d”

Query verso Google:

- termine di ricerca: `“http://www.univaq.it”`
- URL: `http://www.google.com/search?hl=en&lr=&ie=UTF-8&oe=UTF-8&q=http%3A%2F%2Fwww.univaq.it`
- `%3A` corrisponde al carattere `:`
- `%2F` corrisponde al carattere `/`
- in altri termini, `q = http://www.univaq.it`

Elementi /1

- ❑ Gli URL hanno tipicamente la seguente forma:

`<schema>:<percorso>?<query>#<frammento>`

- ❑ non sempre sono tutti presenti (a parte lo schema): alcuni non hanno la query, ad altri manca il frammento, etc.
- ❑ quasi sempre gli URL hanno uno spazio di nomi gerarchico: il path inizia con `"//"` e contiene dei caratteri `"/"` a separare le varie parti

Elementi /2

`<schema>:<percorso>?<query>#<frammento>`

- ❑ il `<percorso>` include:
 - identificativo in rete dell'host che ospita il server web
 - path name della risorsa in qualche supporto di memorizzazione persistente dell'host (es., file system)

- ❑ Dunque, un URL può essere inteso come una estensione, verso la rete, del meccanismo dei nomi gerarchici per le risorse di un file system

- ❑ Domanda: ma il path al punto 2. è relativo o assoluto rispetto al file system locale? Ovvero, come funziona un web server ...?

URL assoluti e relativi

- ❑ Un **URL assoluto** contiene tutte le parti predefinite dal suo schema, esplicitamente precisate
- ❑ Un URL può però anche essere relativo, ed in questo caso riportare solo una parte dell'URL assoluto corrispondente. Sicuramente manca la parte schema.
- ❑ Un **URL relativo** fa sempre riferimento ad un URL di base (ad esempio, l'URL assoluto del documento ospitante l'URL relativo) rispetto al quale fornire elementi di differenza

Risolvere un URL relativo

Significa identificare l'URL assoluto sulla base dell'URL relativo stesso e dell'URL di base:

- ❑ se inizia con "#", è un frammento interno allo stesso documento di base
- ❑ se inizia con uno schema, è un URL assoluto
- ❑ altrimenti, si estrae il path assoluto dell'URL di base, meno l'ultimo elemento, e si aggiunge in fondo l'URL relativo
- ❑ si procede infine a semplificazioni per le sequenze:
 - "./" (stesso livello di gerarchia): viene cancellata
 - "../" (livello superiore di gerarchia): viene eliminato insieme all'elemento precedente
- ❑ ciò che rimane (se rimane qualcosa) è l'URL assoluto risolto. Ogni ostacolo incontrato genera un errore.

Esempi

- Dato l'URL base
`http://a/b/c/d`
i seguenti URL relativi
diventano:

<code>g</code>	<code>http://a/b/c/g</code>
<code>./g</code>	<code>http://a/b/c/g</code>
<code>#s</code>	<code>http://a/b/c/d#s</code>
<code>g#s</code>	<code>http://a/b/c/g#s</code>
<code>.</code>	<code>http://a/b/c/</code>
<code>./</code>	<code>http://a/b/c/</code>
<code>..</code>	<code>http://a/b/</code>
<code>../</code>	<code>http://a/b/</code>
<code>../g</code>	<code>http://a/b/g</code>
<code>../..</code>	<code>http://a/</code>
<code>../.. /</code>	<code>http://a/</code>
<code>../.. /g</code>	<code>http://a/g</code>